

a. Descripción general

i. Nombre del componente

`ConfigPanel`

ii. Propósito y problema que resuelve

`ConfigPanel` es un `JPanel` personalizado que agrupa varios componentes avanzados para crear una interfaz de configuración segura.

Resuelve tres problemas habituales:

- Evitar modificaciones accidentales en listas sensibles.
- Controlar el comportamiento mediante opciones visuales claras.
- Proteger acciones mediante autenticación básica (contraseña).

El usuario solo puede interactuar con la lista cuando activa el “modo seguro” mediante contraseña, y puede elegir cómo se comporta la lista con botones de opción.

iii. Componentes personalizados integrados

Componente	Autor / Origen	Clase
Checkbox con contraseña	Pablo	<code>PasswordCheckBox</code>
RadioButton avanzado	Israel	<code>CustomRadioButton<T></code>
Lista avanzada	Jose	<code>Advanced_JList<E></code>

b. Estructura del componente

i. Organización interna

```
ConfigPanel (BorderLayout)
├─ NORTH: Panel superior
│   ├── JLabel ("Modo de movimiento:")
│   ├── CustomRadioButton ("Mover arriba")
│   └─ CustomRadioButton ("Mover abajo")
│
├─ CENTER:
│   └─ JScrollPane
│       └─ Advanced_JList
│
└─ SOUTH: Panel inferior
    ├── PasswordCheckBox ("Activar modo seguro")
    └─ JLabel (ayuda)
```

ii. Layout utilizado

- `BorderLayout` en el panel principal.
- `FlowLayout` en los paneles superior e inferior.

Esto da una estructura clara: controles arriba, contenido en el centro, seguridad abajo.

c. Propiedades añadidas (en `ConfigPanel`)

El panel encapsula y gestiona:

- Estado de seguridad (a través de `PasswordCheckBox`).
- Modo de desplazamiento de la lista (ascendente / descendente).
- Activación o bloqueo de la lista (`setActive` de `Advanced_JList`).

Aunque no expone getters/setters propios, **orquesta propiedades de componentes personalizados**, convirtiéndose en un componente compuesto reutilizable.

d. Eventos

- PasswordCheckBox

- Usa `ActionListener` para activar o desactivar la lista:

```
secureMode.addActionListener(e -> {  
    list.setActive(secureMode.isSelected());  
});
```

-
- CustomRadioButton

- Usa `ActionListener` para cambiar el modo de movimiento:

```
rbAsc.addActionListener(e -> list.setDesplazamientoDesc(false));  
rbDesc.addActionListener(e -> list.setDesplazamientoDesc(true));
```

-
- Advanced_JList

- Maneja doble clic con `MouseListener` para mover elementos.

Los eventos se encadenan entre componentes, creando comportamiento coordinado.

e. Ejemplo de uso

i. Código

```
public class Test {  
    public static void main(String[] args) {  
        SwingUtilities.invokeLater(() -> {  
            JFrame f = new JFrame("Demo");  
            f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
            f.add(new ConfigPanel());  
            f.setSize(400, 300);  
            f.setLocationRelativeTo(null);  
            f.setVisible(true);  
        });  
    }  
}
```

ii. Capturas de pantalla

(En el documento final puedes incluir dos capturas:)

1. Estado inicial

- Lista desactivada.
- Checkbox sin marcar.

2. Estado activo

- PasswordCheckBox marcado tras introducir contraseña.
 - Lista activa.
 - Doble clic mueve elementos según el radio seleccionado.
-

f. Ficha de valoración por componente

1. PasswordCheckBox

Ítem	Nota	Justificación
Utilidad	9	Muy útil para proteger acciones críticas.
Calidad	8	Código simple, claro y reutilizable.
Documentación	8	Fácil de explicar y entender.
Funcionamiento	9	Hace exactamente lo que promete.

2. CustomRadioButton

Ítem	Nota	Justificación
Utilidad	8	Mejora la interacción respecto a un <code>JRadioButton</code> normal.
Calidad	9	Buen uso de eventos, genérico y extensible.
Documentación	7	Requiere algo de lectura para entender todos los eventos.
Funcionamiento	9	Responde correctamente a clics, doble clic y ratón.

3. Advanced_JList

Ítem	Nota	Justificación
Utilidad	7	Muy concreta, pero práctica en su contexto.
Calidad	7	Correcta, sencilla y estable.
Documentación	6	Poco explícita, se entiende leyendo el código.
Funcionamiento	8	El doble clic funciona de forma coherente.

`ConfigPanel` no es solo un contenedor, es un **componente compuesto** que coordina comportamiento, eventos y estado entre elementos personalizados, demostrando diseño modular en Swing.